Github:

1 Definitioner

Odds: Odds representerar utbetalningsfaktorn för ett vad. Om man placerar ett vad med en insats på 100 kr och ett odds på 2.0 så betalas 200 kr ut vid vinst.

Marknad (Market): En fråga och minst två utfall där varje utfall har ett odds. Exempelvis: "Vem vinner matchen?" (fråga) & "Lag A (2.0), Lag B (2.0), Oavgjort (1.25)" (utfall)

Vad (Bet): Ett utfall på en marknad, en insats och oddsen att utfallet inträffar. Exempelvis: "Jag satsar 100kr (insats) på att lag A (utfall) vinner matchen med ett odds på 2.5".

Kombinationsspel (Bet with legs): En typ av vad som består av flera ”undervad” (legs). Om man vinner alla undervad så vinner man kombinationsspelet, annars förlorar man det. Oddsen för ett kombinationsspelet är lika med produkten av oddsen för alla vad som ingår i spelet. De olika undervaden måste vara okorrelerade. Exempelvis: "Jag slår vad om att lag A vinner fotbollsmatch 1 till ett odds på 2 OCH att lag B vinner fotbollsmatch 2 till ett odds på 1.5 för en kombinerad odds på 2 * 1.5 = 3"

Självexkludering: En tidsperiod då användaren inte får slå vad eller göra insättningar.

GDPR-radering: Information i systemet som identifierar en person raderas så länge det inte bryter mot andra lagkrav.

2 Krav

2.1 Funktionella krav

2.1.1. Autentisering

2.1.1.1. Systemet skall kunna verifiera en användares identitet.
2.1.1.2. Systemet skall endast tillåta en användare per person.
2.1.1.3. Besökare skall kunna registrera sig själva i systemet.
2.1.1.4. Systemet skall vid registrering av användare verifiera att personen är myndig.
2.1.1.5. Användare skall kunna logga in i systemet.
2.1.1.6. Användare skall kunna logga ut ur systemet.

2.1.2 Penningrelaterat

2.1.2.1. Användare skall kunna föra över pengar digitalt från ett bankkonto som användaren äger.
2.1.2.2. Användare skall kunna föra över pengar digitalt från sitt systemkonto till ett bankkonto som användaren äger.
2.1.2.3. Användare skall kunna se saldo på sitt konto.
2.1.2.4. Användare skall kunna se historik över insättningar, uttag och vad.

2.1.3 Spelgränser

2.1.3.1. Användare skall kunna sätta periodbaserade insättningsgränser (t.ex. maxinsättningsbelopp per 30 dagar = 1000kr).

2.1.3.2. Användare skall kunna sätta periodbaserade förlustgränser (t.ex. max summa förlust per 30 dagar = 1000kr).

2.1.3.3. Systemet skall garantera att penningrelaterade gränser inte överskrids.

2.1.3.4. Användare skall kunna sätta upp självexkludering genom att specificera en tidsperiod.

2.1.4 Användargränssnitt

2.1.4.1. Man skall kunna interagera med systemet via ett grafiskt användargränssnitt.

2.1.5 Loggning

2.1.5.1. Systemet skall logga alla administrativa åtgärder.

2.1.5.2. All information som används för att avgöra vad och marknader skall loggas.

2.1.6 Vadslagning

2.1.6.1. Användare skall, förutsatt att användaren har tillräckligt med saldo, kunna placera vad (inklusive kombinationsspel) på marknader.

2.1.6.2. Systemet skall automatiskt avgöra om ett vad är vunnet eller förlorat när marknadens utfall är känt.

2.1.6.3. När ett vad har vunnits så skall systemet automatiskt kreditera användarens konto med vinsten.

2.1.7 Statistik

2.1.7.1. Systemet skall kunna presentera statistik om marknader via ett grafiskt användargränssnitt. Mätvärden: omsättning, antal vad. Dimensioner: händelse, typ av händelse.

2.1.8 GDPR

2.1.8.1. Användare skall kunna begära radering av sitt konto enligt GDPR.

2.1.9 Marknader

2.1.9.1. Systemet skall kunna skapa nya marknader.

2.1.9.2. Systemet skall kunna modifiera existerande marknader.

2.1.9.3. Aktörer skall, förutsatt att de har tillräckliga rättigheter, kunna uppdatera oddsen för utfall på marknader.

2.1.9.4. Aktörer skall kunna specificera om odds på ett utfall får uppdateras av automatiska processer.

2.1.9.5. Systemet skall hindra nya vad från att placeras på marknader som är avgjorda.

2.1.10 Fusk

2.1.10.1. Systemet skall flagga användare som totalt har lagt fler än X vad och som totalt har vunnit fler vad än den Y:te percentilen. X & Y skall vara konfigurerbara.

2.1.11 Förbättringsförslag

2.1.11.1. Användare skall kunna skicka in förbättringsförslag.

2.1.11.2. Systemet skall automatiskt upptäcka om två förbättringsförslag har identiskt semantiskt innehåll.

2.1.11.3. Systemet skall kunna skapa upp ärenden i ett externt ärendehanteringssystem för varje semantiskt unikt förbättringsförslag.

2.2 Icke-funktionella krav

2.2.1. Säkerhet

2.2.1.1. All kommunikation in och ut från systemet skall vara säkert krypterad.

2.2.1.2. All information i vila skall vara säkert krypterad.

2.2.2 Lagkrav

2.2.2.1. Det skall inte ta mer än 30 dagar från det att en användare har begärt radering enligt GDPR tills dess att systemet har utfört raderingen.

2.2.3 Användbarhet

2.2.3.1. Användargränssnittet skall vara tillgängligt via moderna webbläsare (senaste 3 versionerna för de 5 största webbläsarna) på datorer, surfplattor & smarta telefoner.

2.2.3.2. Systemet skall visa felmeddelanden när en process som användaren har påbörjat inte går att slutföra. Felmeddelandet skall med mindre än 300 karaktärer beskriva att det inte gick att slutföra processen, vilken process som inte gick att sluföra, varför det inte gick att slutföra processen, och vad användaren kan göra för att processen ska lyckas (om denne kan göra något).

2.2.4 Prestanda

2.2.4.1. Användargränssnittet skall, efter att användaren har interagerat med det, uppdateras inom 150 millisekunder på enheter som är högst 2 år gamla. Det innebär endast att användargränssnittet bör vara responsivt, INTE att samtliga processer som startas av användaren skall slutföras inom 150 millisekunder.

2.2.4.2. Om information som presenteras i användargränssnittet ändras, så ska det inte ta mer än 1 sekund innan användargränssnittet reflekterar den uppdaterade informationen.

2.2.4.3. Systemet skall inom 2 sekunder från det att en händelse har inträffat ha uppdaterat oddsen för relaterade marknader.

2.2.5 Tillgänglighet

2.2.5.1. Systemet skall ha en tillgänglighet på 99.9% per år.

3 Use case

3.1 Besökare skapar konto

Aktör: Besökare
Mål: Att bli medlem och kunna spela
Trigger: Klicka "Bli medlem"

Huvudflöde

1. Aktören väljer autentiseringsmetod.
2. Aktören identifierar sig med vald autentiseringsmetod
3. Aktören verifierar sin identitet.
4. Systemet verifierar att personen inte redan har ett konto.
5. Systemet verifierar att personen är myndig.
6. Aktören skriver in e-postadress under kontaktuppgifter.
7. Aktören accepterar villkor och integritetspolicy.
8. Aktören klickar "Bli medlem".
9. Systemet skickar ett välkomstmejl till e-postadressen.
10. Aktören omdirigeras till en sida som möjliggör insättning av pengar.

Alternativflöden

1. Personen är inte myndig: Ett felmeddelande visas.
2. Personen har redan ett konto: Ett felmeddelande visas.

3.2 Besökare loggar in

Aktör: Besökare
Mål: Att logga in och få tillgång till medlemsfunktioner
Trigger: Klicka "Logga in"

Huvudflöde

1. Aktören väljer en autentiseringsmetod.
2. Aktören identifierar sig med vald autentiseringsmetod.
3. Aktören omdirigeras till landningssidan.

Alternativflöden

1. Användaren har inget konto: Systemet visar ett felmeddelande och en knapp "Registrera Dig".

3.3 Kund inspekterar resultatet av ett avgjort vad

Aktör: Kund
Mål: Att få insyn i resultatet av ett avgjort vad
Trigger: Klicka på "Min spelhistorik"

Huvudflöde

1. Aktören omdirigeras till sidan som visar spelhistorik i omvänd kronologisk ordning.
2. Aktören skrollar till eller söker med fritext efter det efterfrågade vadet.

3.4 Kund sätter in pengar

Aktör: Kund
Mål: Att kunna spela för riktiga pengar
Trigger: Klicka på "Sätt in pengar" eller efter att ett konto har skapats.

Huvudflöde

1. Aktören väljer betalningsmetod.
2. Aktören väljer ett belopp att sätta in.
3. Systemet kontrollerar att insättningen inte överskrider insättningsgränsen.
4. Aktören sätter in pengar via vald betalningsmetod genom att följa betalningsflödet.
5. Ett meddelande visas som bekräftar att pengarna har överförts.
6. Aktören omdirigeras till landningssidan.

Alternativflöden

1. Misslyckad betalning: Ett meddelande visas. Aktören kan välja en annan betalningsmetod eller avbryta insättningen.
2. Insättningen överskrider insättningsgräns: Ett felmeddelande visas.

3.5 Kund ställer in spelpaus

Aktör: Kund
Mål: Att ställa in spelpaus som hindrar kunden från att spela under en viss tidsperiod
Trigger: Klicka på "Ställ in spelpaus"

Huvudflöde

1. Aktören väljer intervall (dag, vecka, månad, år) och längd (en positiv siffra).
2. Aktören klickar "Ställ in spelpaus".
3. Systemet säkerställer att spelpausen inte förkortar en existerande spelpaus.
4. Aktören möts av ett meddelande som sammanfattar spelpausen och kräver ytterligare en bekräftelse (Ja/Nej).
5. Aktören omdirigeras till landningssidan.

Alternativflöden

1. Aktören väljer att inte bekräfta spelpausen i steg 3: meddelandet försvinner men aktören omdirigeras inte.

3.6 Kund tar ut pengar

Aktör: Kund
Mål: Att ta ut pengar
Trigger: Klicka på "Ta ut pengar"

Huvudflöde

1. Aktören väljer utbetalningsmetod.
2. Aktören väljer utbetalningsbelopp.
3. Systemet kontrollerar att kontot har tillräckligt saldo.
4. Aktören följer det valda utbetalningsflödet.
5. Aktören får information som bekräftar att en överföring av pengarna har påbörjats.
6. Aktören klickar "Stäng" för att stänga informationen och återvända till utbetalningssidan.

Alternativflöden

1. Kunden har inte tillräckligt med saldo: ett felmeddelande visas.

3.7 Kund satsar pengar på ett vad

Aktör: Kund
Mål: Att satsa pengar på ett vad
Trigger: Klicka på ett av utfallen under marknadsbeskrivningen.

Huvudflöde

1. Aktören får upp en ruta som innehåller en beskrivning av vadet och en inmatningsruta där insatsen kan anges.
2. Aktören anger en insats.
3. Aktören klickar "Placera vad".
4. Systemet kontrollerar att aktören får spela.
5. Systemet kontrollerar att aktören har tillräckligt med pengar.
6. Systemet kontrollerar att marknaden inte är avgjord.
7. Vadet placeras, pengarna dras från aktörens konto och ett kvitto presenteras.

Alternativflöden

1. Otillräckligt saldo: en informationsruta visas som beskriver problemet samt innehåller en knapp "Sätt in pengar".

3.8 Kund sätter spelgränser

Aktör: Kund
Mål: Att begränsa sitt spelande genom insättnings- och förlustgränser
Trigger: Klicka på "Spelgränser"

Huvudflöde

1. Aktören omdirigeras till sidan för spelgränser.
2. Aktören anger en period (t.ex. dag, vecka, månad).
3. Aktören anger en insättningsgräns.
4. Aktören anger en förlustgräns.
5. Aktören klickar "Spara".
6. Systemet sparar gränserna och visar ett meddelande som bekräftar att spelgränserna har uppdaterats.

3.9 Kund ser kontosaldo

Aktör: Kund
Mål: Att se sitt aktuella kontosaldo
Trigger: Aktören öppnar kontoöversikten

Huvudflöde

1. Aktören omdirigeras till kontoöversikten.
2. Systemet visar aktuellt kontosaldo.

3.10 Administratör uppdaterar odds

Aktör: Administratör
Mål: Att uppdatera oddsen för ett utfall
Trigger: Aktören väljer en existerande marknad

Huvudflöde

1. Aktören öppnar en marknad.
2. Aktören ändrar oddsen för ett eller flera utfall.
3. Aktören klickar "Spara".
4. Systemet uppdaterar oddsen för marknaden.

3.11 Kund begär GDPR-radering

Aktör: Kund
Mål: Att få sitt konto raderat enligt GDPR
Trigger: Klicka på "Radera konto"

Huvudflöde

1. Aktören begär radering av sitt konto.
2. Systemet säkerställer att aktören inte har några öppna vad och att aktörens saldo är lika med noll.
3. Systemet visar information om vad raderingen innebär.
4. Aktören bekräftar raderingen.
5. Systemet registrerar begäran och påbörjar raderingsprocessen.
6. Systemet visar ett meddelande som bekräftar att begäran har registrerats.

3.12 Kund skickar förbättringsförslag

Aktör: Kund
Mål: Att skicka in ett förbättringsförslag för systemet
Trigger: Klicka på "Skicka förbättringsförslag"

Huvudflöde

1. Aktören skriver ett förbättringsförslag i ett textfält.
2. Aktören klickar "Skicka".
3. Systemet analyserar förslaget för att avgöra om det är semantiskt unikt.
4. Om förslaget är unikt skapar systemet ett ärende i ett externt ärendehanteringssystem.
5. Systemet visar ett meddelande som bekräftar att förslaget har skickats.

4. Objektorienterad analys och domänmodell

Ett urval av de centrala begreppen i domänen är:

1. Kund (Customer)
   Typ: Aggregate
   Ansvar: Säkerställa att ett urval av all kundrelaterad information uppfyller och ändras enligt affärsregler. Urvalet består av personuppgifter och uppgifter om ”ansvarsfullt spelande” (t.ex. insättningsgränser & självexkludering).
   Analys: Kunden består av informationen: CustomerId, CustomerIdentity (innehåller personuppgifter, kanske dåligt namn), ResponsibleGambling (innehåller uppgifter om gränser för ansvarsfullt spelande) och Account (sköter transaktioner). Det finns mer kundrelaterad information än så, t.ex. alla vad kunden har lagt, men i praktiken blir det för mycket data att hämta upp varje gång kunden ska modifieras, så vi bryter ut det i separata aggregates. Det är en DDD-regel: aggregates ska vara små.

2. Konto (Account)
   Typ: Entitet
   Ansvar: Säkerställa att transaktioner skapas enligt affärsregler, t.ex. att balansen på kontot aldrig understiger noll.
   Analys: Ett konto består av: AccountId, Currency, Balance, Transactions. Inget konstigt.

3. Transaktion (Transaction)
   Typ: Entitet
   Ansvar: Att representera en transaktion.
   Analys: En transaktion består av: Amount, TransactionType (Credit/Debit), TransactionReason (ExternalTransfer, Refund etc.) & Timestamp. Inget konstigt.

4. Marknad (EventMetricMarket)
   Typ: Aggregate
   Ansvar: Att beskriva vad en marknad är och hur marknader kan förändras.
   Analys: Det är svårt att representera alla möjliga marknader med en klass, men det är möjligt att representera många med ett fåtal klasser. Min definition av en marknad utgår ifrån följande antaganden: det finns ALLTID EXAKT en första tidpunkt då marknaden kan avgöras, det finns ALLTID minst 2 möjliga utfall, marknaden avgörs ALLTID av EXAKT 1 mätvärde som kan fångas på eventnivå (t.ex. antal mål gjorda). Det sista antagandet stämmer inte: vissa marknader kan baseras på flera mätvärden (t.ex. ”blir antal hörnor + antal mål över eller under 7 i först halvlek”), men vi begränsar oss för enkelhetens skull. Utöver detta så innehåller marknader också logik som bestämmer om marknaden är avgjord givet information om eventet (TrySettle(EventData)). Det finns en klass som ärver direkt från EventMetricMarket och den heter ScopedEventMetricMarket. ScopedEventMetricMarket gör det möjligt att definiera marknader som gäller ett visst mätvärde på en viss aggregeringsnivå: spelare, lag eller totalt. Sedan finns det tre konkreta klasser som ärver från dessa: ComparisonScopedEventMetricMarket, EqualScopeEventMetricMarket, OptimalScopedEventMetricMarket. ComparisonScopedEventMetricMarket kan uttrycka marknader på form ”är mätvärdet X större än, mindre än, eller lika med ett referensvärde vid tidpunkt T (JA/NEJ)”. Kan användas för att slå vad om hur ett mätvärde kommer förändras. EqualScopeEventMetricMarket kan uttrycka marknader som följer formen ”är mätvärdet X för alla lag i matchen lika vid tidpunkt T”. Syftet är att kunna uttrycka ”oavgjort”-marknader. OptimalScopedEventMetricMarket kan uttrycka marknader som följer formen ”är mätvärdet X större än/mindre än alla andra mätvärden på samma aggregeringsnivå vid tidpunkt T? (JA/NEJ)”. Syftet är att kunna uttrycka vinnaren av ett event.

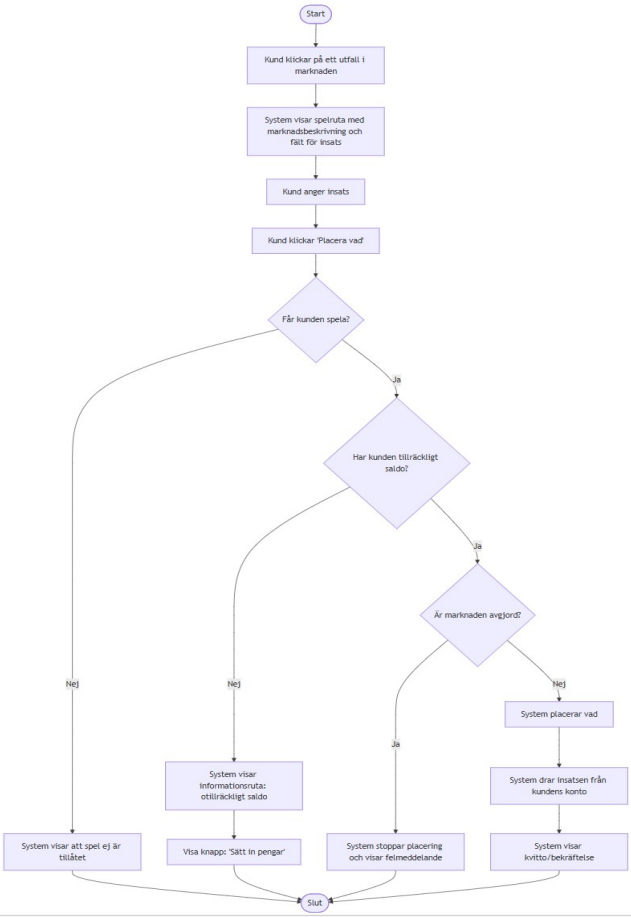
5. Vad (Bet)
   Typ: Aggregate
   Ansvar: Att beskriva vad ett vad är och hur vad kan förändras.
   Analys: Ett vad består av en kundidentifierare, ett eller flera undervad (bet legs, kombinationsspel), och en insats. Det är en bred definition.

6. Undervad (BegLeg)
   Typ: Entitet
   Ansvar: Att representera ett ”undervad”.
   Analys: Ett undervad består av en marknadsidentifierare, en utfallsidentifierare, ett odds och en status.

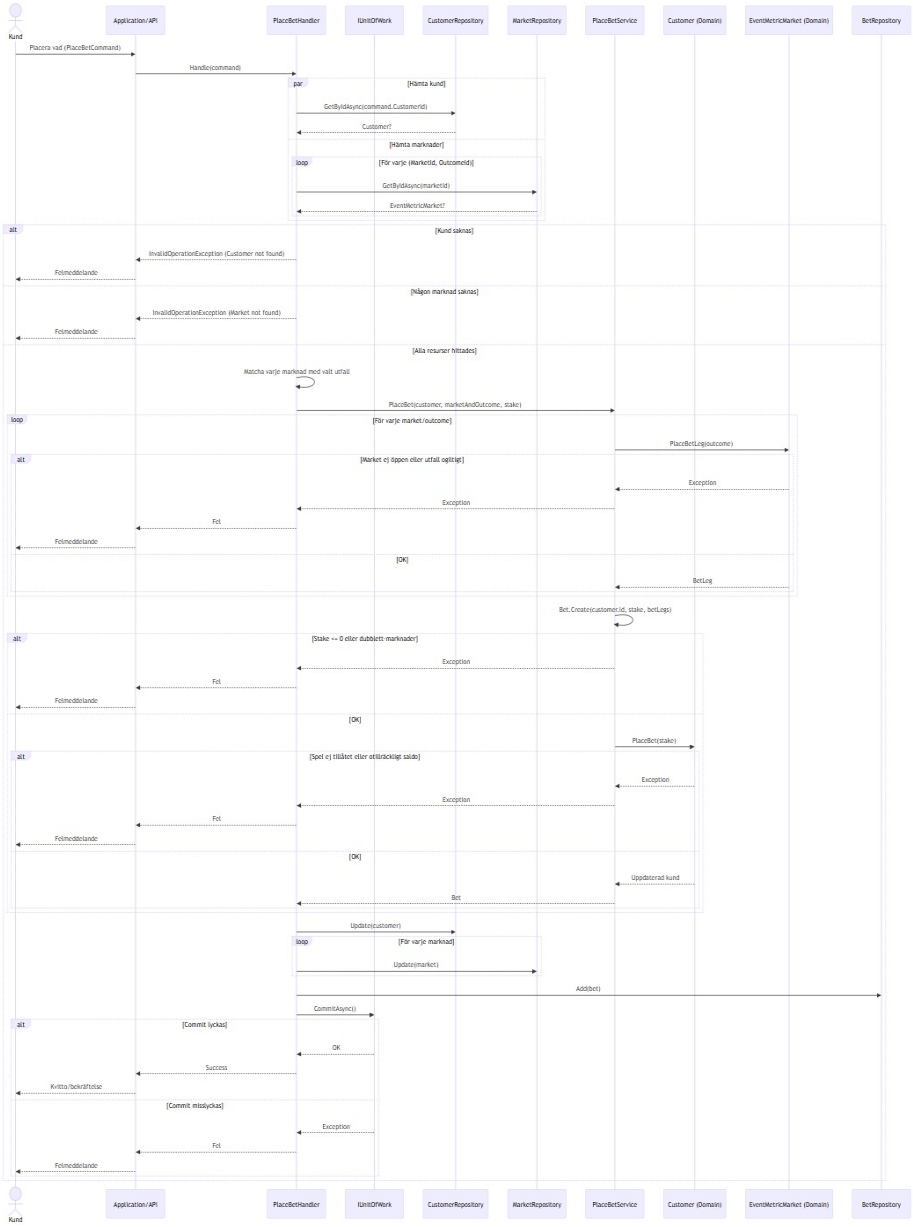
7. Event (Event)
   Typ: Aggregate
   Ansvar: Att beskriva vad ett event är och hur event kan förändras.
   Analys: I min definition av ett event så består det av: startdatum, status (scheduled/inProgress/finished/cancelled), en samling mätvärden, och minst 1 lag med minst 1 spelare. Det är en bred definition.

8. Tidsstämpel (Timestamp)
Typ: Value Object
Analys: När vi definierar marknader så behöver vi kunna uttrycka NÄR marknaden kan avgöras. Det gör vi med tidsstämplar. Dessa tidsstämplar har en metod ”bool HasOccurred(EventData)” som avgör om tidsstämpeln har inträffat givet all information om ett event. Syftet med att basera tidsstämplarna på information om eventet är att man ska kunna uttrycka händelserbaserade tidsstämplar som t.ex. ”nästa gång någon gör ett mål” eller ”nästa gång eventets status ändras till 'avslutat'”.
Ansvar: Att representera en tidsstämpel, som innefattar att kunna avgöra huruvida tidsstämpeln har inträffat givet information om en händelse.

Aktivitetsdiagram use case 3.7 ”Kund placerar vad”



Sekvensdiagram use case 3.7 ”Kund placerar vad”



5. Systemdesign och arkitektur

Vi börjar med att dela in systemet i två delsystemet: analytics & operations.

Analytics

Ansvarar för att möjliggöra analys på stora mängder data från olika källor i beslutsstödjande syfte. Systemet består av två databaser: en OLAP-databas (som kan lagra och räkna på stora mängder data) & en OLTP-databas (som används för att tracka integrationernas tillstånd).

Operations

Ansvarar för att möjliggöra vadslagning för kunder i vinstsyfte.

Det finns två databaser i systemet: en OLAP-databas, och en krypterbar relationell OLTP-databas. OLAP-databasens syfte är att lagra (väldigt stora mängder) information om historiska events och beräkna sannolikheten för att olika utfall inträffar. OLTP-databasens syfte är att lagra all annan information som t.ex. kunder, vad, marknader, transaktioner, events.

Det finns två presentationsapplikationer i systemet: ett för kunder & ett för personal. Båda presentationsapplikationerna: är SPA:er, använder sig av websockets för att snabbt kunna presentera uppdateringar, använder sig av BankID för autentisering.

Systemet består av en monolitisk, horisontellt & vertikalt skalbar serverapplikation. Applikationen: har ett API, stödjer endast HTTPS med TLS 1.2.

Systemet integrerar följande system: BankID (autentisering), Stripe (betalningar), Stripe Connect (uttag), Sportradar (live-uppdateringar från sportevent), ChatGPT (för att kunna upptäcka förslagdubbletter), SendGrid (skicka mejl).

På kodnivå så delas systemet in i fyra moduler: domän, applikation, infrastruktur och presentation. Domänlagret ansvarar för att definiera domänmodeller och implementera affärslogik. Applikationslagret ansvarar för att orkestrera use case som innefattar upphämtning av domänmodeller från databasen, anropa domänlogik, och spara ner modifierade domänmodeller. Infrastrukturlagret implementerar de beroenden som domänlagret & applikationslagret har definierat. Presentationslagret innehåller serverapplikationen & SPA:erna. Domänlagret är endast beroende av programspråket och dess inbyggda funktioner.

6. Designmönster och designprinciper

Unit of Work: Används i applikationslagret när ett use case (PlaceBet) kräver att flera aggregat modifieras transaktionellt.

Factory: Används vid registrering av kund (domainservice CustomerRegistrationService) då registreringsprocessen måste följa diverse policies. Man skulle kunna lägga registrering direkt i Customer.Register men då rör man sig bort från single-responsibility principle.

Policy: Används vid registrering av kund (domainservice CustomerRegistrationService, IAgeCustomerRegistrationPolicy) för att säkerställa att kundens ålder uppfyller ett villkor. Tanken är att policyn ska implementeras av applikationslagret så att man kan konfigurera villkoret för att kunna köra i flera länder utan att skriva om.

Repository: T.ex. ICustomerRepository.

Domain service: T.ex. PlaceBetService som måste modifiera flera aggregat (customer, market, bet). Den funktionaliteten passar m.a.o inte i ett aggregat så den bryts ut i en domain service.

Single responsibility principle: Används i Domain/Aggregates/Customer.cs. Istället för att ha alla affärsregler i customer har customer delats in i klasserna CustomerIdentity/ResponsibleGambling/Account.

Dependency inversion principle: Används i Domain/Services/CustomerRegistrationService.cs. Domänlagret definierar policyn, något annat lager implementerar den, och applikationslagret ger domänlagret en instans av policyn. Används även i applikationslagret i Application/Abstractions/Repositories/IBetRepository.

Liskov substitution principle: Används överallt där EventMetricMarket (Domain/Aggregates/Markets/Abstractions/EventMetricMarket.cs) används, då EventMetricMarket är en abstrakt klass.

Kodexempel:

1. Domain/Services/PlaceBetService.cs
2. Domain/Services/CustomerRegistrationService.cs